Лабораторная работа №1.

Первое знакомство с Docker

Тема: Запуск и исследование контейнеров. Nginx и жизненный цикл Docker

Цель: Освоить запуск контейнеров из готовых образов, проверку работы Docker Engine, диагностику Nginx и управление жизненным циклом контейнеров.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №1. Первое знакомство с Docker. Продолжительность: 4 академических часа (180 минут). Работа проводится индивидуально в Debian 13 с предварительно установленным Docker Engine. Все команды выполняются в терминале Debian; в примерах для доступа к Docker используется sudo.

Работа закрепляет лекцию №1 «Введение в контейнеризацию. Архитектура Docker». Установка Docker Engine в данной лабораторной не рассматривается: это тема следующего занятия.

2. Тема, цель и результаты

Тема: запуск готовых Docker-образов, исследование состояния контейнеров, публикация порта веб-сервера Nginx, просмотр логов и управление жизненным циклом контейнеров.

Цель: научиться проверять доступность Docker Daemon, создавать контейнеры из готовых образов и проводить базовую диагностику их работы.

1. Определять версии Docker Client и Docker Server и состояние службы Docker.
2. Объяснять различие между Docker Image и Docker Container.
3. Запускать и исследовать контейнеры hello-world и Nginx.
4. Проверять отображение веб-страницы и публикацию порта.
5. Просматривать логи, выполнять команды в контейнере и исследовать его конфигурацию.
6. Останавливать, повторно запускать и удалять контейнеры, не удаляя образ.

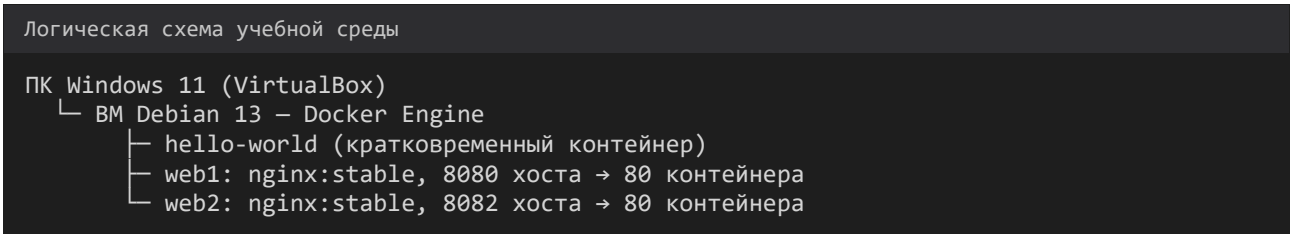
3. Оборудование и программное обеспечение

- ПК Windows 11 с VirtualBox либо компьютер с Linux.
- Виртуальная машина Debian 13 с настроенным сетевым доступом и Docker Engine.
- Терминал Debian с возможностью выполнять sudo.
- Браузер Debian либо утилита curl (если имеется).
- Доступ к реестру образов для первоначальной загрузки hello-world и nginx:stable.

Важно: localhost внутри Debian обозначает Debian, а не Windows-хост. Если браузер запускается на Windows, необходимо обеспечить сетевую доступность адреса Debian-ВМ; варианты зависят от режима сети VirtualBox. Все обязательные проверки в работе можно выполнить из самой Debian.

4. Исходная схема и данные

Логическая схема учебной среды



Имена и порты контейнеров

Контейнер	Образ	Порт хоста	Порт контейнера
hello-world	hello-world	—	—
web1 (этап 3)	nginx:stable	8080	80
web1 (этап 6)	nginx:stable	8081	80
web2 (этап 6)	nginx:stable	8082	80
student-web	nginx:stable	8090	80

Порты 8080, 8081, 8082 и 8090 должны быть свободны на Debian. Для предотвращения конфликтов выполняйте этапы последовательно и очищайте собственные контейнеры после проверки.

5. Подготовка среды

Проверьте, что вы работаете именно в терминале Debian. Пользователь должен иметь право применять sudo. Не добавляйте пользователя в группу docker ради выполнения этой лабораторной: членство в такой группе даёт широкие полномочия на хосте.

```
Проверка установленной клиентской программы
docker --version
```

Команда показывает версию клиента Docker. Если команда не найдена, сообщите преподавателю: в этой работе установка Docker не выполняется. Критерий продолжения: отображается версия Docker.

```
Проверка службы Docker Engine
sudo systemctl status docker
```

Команда запрашивает состояние системной службы. Найдите строку Active: active (running). Для выхода из просмотра нажмите q. Если служба неактивна, обратитесь к преподавателю; не изменяйте системную конфигурацию самостоятельно.

```
Проверка связи клиента с сервером
sudo docker version
```

В результате должны отображаться разделы Client и Server. Если присутствует только Client и ошибка соединения, клиент не может обратиться к Docker Daemon. Запишите сообщение об ошибке.

```
Общие сведения об окружении Docker
sudo docker info
```

```

```

Найдите поля Server Version, Containers, Images, Storage Driver, Operating System и Architecture. Значения зависят от установленной системы. Запишите их в отчёт.

6. Этап 1. Первый контейнер hello-world

Создайте и запустите контейнер из готового тестового образа. При отсутствии образа локально Docker загрузит его из реестра.

```
Создание и запуск тестового контейнера
sudo docker run hello-world
```

docker run создаёт контейнер из образа и запускает основной процесс. Ожидаемый результат: в терминале появляется текст Hello from Docker! и информация об успешной работе. Основной процесс завершится — контейнер не будет продолжать работать.

```
Просмотр работающих контейнеров
sudo docker ps
```

ps без ключей показывает только работающие контейнеры. Завершившийся hello-world в этом списке отсутствует. Это ожидаемое поведение.

```
Просмотр всех контейнеров
sudo docker ps -a
```

Ключ -a показывает и остановленные экземпляры. Найдите hello-world со статусом Exited. Если работа запускалась неоднократно, возможны несколько контейнеров из одного образа.

```
Просмотр локальных образов
sudo docker images
```

Команда показывает загруженные образы. Убедитесь, что hello-world присутствует в списке. Критерий завершения этапа: образ сохранён локально, а тестовый контейнер завершил выполнение.

В отчёте объясните: почему hello-world отсутствует в docker ps, но присутствует в docker ps -a? Почему удаление контейнера не эквивалентно удалению образа?

7. Этап 2. Запуск веб-сервера Nginx

Перед запуском убедитесь, что контейнер с именем web1 не существует. Проверка позволяет избежать конфликта имён.

```
Проверка существующих имён контейнеров
sudo docker ps -a
```

Если web1 уже существует после предыдущей попытки, сначала установите его статус и при необходимости согласуйте удаление собственного тестового контейнера с преподавателем.

```
Запуск веб-сервера Nginx

sudo docker run -d --name web1 -p 8080:80 nginx:stable
```

Ключ -d запускает контейнер в фоновом режиме. --name web1 задаёт имя. -p 8080:80 публикует порт Debian 8080 и направляет трафик на порт 80 внутри контейнера. nginx:stable — имя образа с тегом. При успешном запуске Docker выведет идентификатор созданного контейнера.

```
Контроль состояния Nginx

sudo docker ps
```

Найдите web1 со статусом Up и публикацией порта 8080→80. Если контейнер сразу остановился либо команда сообщила об ошибке, запишите сообщение и проверьте конфликты имён или портов.

Откройте в браузере Debian адрес http://localhost:8080. При успехе появится стандартная страница Welcome to nginx!.

```
Альтернативная HTTP-проверка при наличии curl

curl -I http://localhost:8080
```

Ключ -I запрашивает заголовки HTTP-ответа. Нормальный результат — статус 200 OK. Если curl не установлен и браузер доступен, используйте браузер; необязательные пакеты не требуется устанавливать ради проверки.

Критерий завершения этапа: Nginx имеет статус Up и отвечает на HTTP-запрос через порт Debian 8080.

8. Этап 3. Исследование работающего контейнера

```
Просмотр журналов Nginx

sudo docker logs web1
```

Команда читает вывод контейнера. После обращения к веб-серверу в журнале обычно присутствует запись HTTP-запроса. Повторите открытие страницы и вновь проверьте журнал.

```
Выполнение команды внутри контейнера

sudo docker exec web1 hostname
```

docker exec запускает дополнительный процесс в уже работающем контейнере. hostname показывает имя хоста, видимое из контейнера. Это не создаёт новый контейнер.

```
Просмотр данных о пользовательском окружении образа
```

```
sudo docker exec web1 cat /etc/os-release
```

Вывод показывает сведения об ОС пользовательского пространства внутри образа. Они не означают наличие отдельного ядра Linux в контейнере.

Запуск интерактивной оболочки

```
sudo docker exec -it web1 /bin/sh
```

Ключ -i сохраняет стандартный ввод, -t выделяет терминал. /bin/sh запускает оболочку; bash может отсутствовать в минимальном образе. Выполните следующие команды внутри контейнера.

Действия внутри интерактивной оболочки

```
pwd
ls
ls /etc/nginx
hostname
exit
```

pwd выводит рабочий каталог, ls показывает файлы, ls /etc/nginx — файлы конфигурации Nginx, hostname — имя хоста контейнера. exit завершает лишь оболочку, запущенную docker exec; основной процесс Nginx продолжает работать.

Исследование конфигурации контейнера

```
sudo docker inspect web1
```

inspect выводит структуру JSON. Найдите Id, Name, State, Image, NetworkSettings и HostConfig.PortBindings. Запишите значение опубликованного порта. Полный вывод может быть большим: в отчёте достаточно скриншота или фрагмента, подтверждающего нужные поля.

Критерий завершения: после выхода из /bin/sh контейнер web1 по-прежнему Up и веб-страница доступна.

9. Этап 4. Жизненный цикл контейнера

Остановка основного процесса контейнера

```
sudo docker stop web1
```

Ожидаемый результат: команда выводит имя контейнера. stop останавливает работу, но не удаляет созданный контейнер.

Проверка после остановки

```
sudo docker ps -a
```

Найдите web1 со статусом Exited. Обновите страницу http://localhost:8080: после остановки контейнер не должен отвечать на запрос.

```
Повторный запуск существующего контейнера

sudo docker start web1
```

Команда запускает тот же экземпляр web1, а не создаёт новый. Проверьте, что статус снова Up.

```
Проверка после повторного запуска

sudo docker ps
curl -I http://localhost:8080
```

Если curl присутствует, ожидается HTTP 200 OK; также допустима проверка через браузер. Сравните имя контейнера и его идентификатор до и после start — они должны остаться прежними.

```
Остановка и удаление учебного контейнера

sudo docker stop web1
sudo docker rm web1
```

docker rm удаляет остановленный контейнер, но не его образ. Команды применяйте только к web1, созданному в этой работе.

```
Контроль очистки контейнера и сохранности образа

sudo docker ps -a
sudo docker images
```

web1 отсутствует в списке контейнеров, а nginx:stable остаётся в перечне локальных образов. Это обязательный вывод этапа.

10. Этап 5. Два контейнера из одного образа

Создайте одновременно два экземпляра образа nginx:stable. Они должны иметь разные имена и не конфликтующие порты хоста.

```
Первый независимый веб-сервер

sudo docker run -d --name web1 -p 8081:80 nginx:stable
```

```
Второй независимый веб-сервер

sudo docker run -d --name web2 -p 8082:80 nginx:stable
```

Оба контейнера работают на внутреннем порту 80; на Debian им соответствуют разные опубликованные порты. Docker должен вернуть отдельные идентификаторы контейнеров.

```
Проверка двух экземпляров

sudo docker ps
```

Проверьте, что web1 и web2 имеют статус Up, а порты хоста указаны как 8081 и 8082. Затем откройте в браузере http://localhost:8081 и http://localhost:8082.

```
Диагностический эксперимент: остановка только web1

sudo docker stop web1
```

После остановки web1 страница на 8081 должна стать недоступной, тогда как 8082 продолжает работать. Это доказывает независимость жизненных циклов двух контейнеров.

```
Восстановление после диагностического эксперимента

sudo docker start web1
```

Проверьте повторную доступность страниц 8081 и 8082. Тем самым эксперимент полностью обратим.

```
Очистка двух тестовых контейнеров

sudo docker stop web1 web2
sudo docker rm web1 web2
```

Критерий завершения: оба учебных контейнера удалены, но образ nginx:stable остаётся локально.

11. Дополнительная диагностика: ошибка конфликта порта

Этот эксперимент выполняется только при успешном завершении предыдущих этапов. Он показывает, что два разных контейнера нельзя одновременно опубликовать на одном и том же порту и адресе хоста.

```
Создайте первый контейнер для опыта

sudo docker run -d --name port-test-1 -p 8091:80 nginx:stable
```

Убедитесь через docker ps, что контейнер запущен и занимает порт Debian 8091.

```
Попытка занять тот же порт хоста

sudo docker run -d --name port-test-2 -p 8091:80 nginx:stable
```

Ожидается ошибка публикации порта. В зависимости от версии Docker неудачный контейнер port-test-2 может остаться в состоянии Created. Запишите точное сообщение ошибки — не требуется запоминать его дословно.

```
Проверка состояния после ошибки
```

```
sudo docker ps -a
```

Сравните статусы двух контейнеров. Причина проблемы — конфликт порта 8091 на Debian, а не внутреннего порта Nginx 80.

```
Восстановление после эксперимента
sudo docker rm -f port-test-1 port-test-2
```

Команда принудительно останавливает и удаляет только два тестовых контейнера. Не применяйте docker rm -f к чужим контейнерам. При отсутствии одного из контейнеров Docker может сообщить, что такой объект не найден; это допустимо.

Критерий завершения: контейнеры port-test-1 и port-test-2 отсутствуют в выводе docker ps -a, порт 8091 свободен.

12. Индивидуальное задание

7. Создайте из nginx:stable контейнер с именем student-web.
8. Опубликуйте внутренний порт 80 контейнера на порту Debian 8090.
9. Подтвердите состояние Up и доступность http://localhost:8090.
10. Найдите в docker inspect имя контейнера и сведения о публикации порта.
11. Остановите контейнер и проверьте недоступность страницы.
12. Запустите тот же контейнер повторно и восстановите доступ к странице.
13. Остановите и удалите контейнер.
14. Проверьте, что образ nginx:stable остаётся в Docker, и составьте таблицу минимум из пяти команд с назначением.

Команды индивидуального задания студент выбирает самостоятельно, опираясь на изученные этапы. Не используйте другое имя или порт без согласования с преподавателем.

13. Итоговая проверка и ожидаемые результаты

15. Docker Client соединяется с Docker Daemon, а служба Docker запущена.
16. Контейнер hello-world успешно выводил приветствие и завершился.
17. Запущенный Nginx обслуживал страницу через опубликованный порт.
18. docker logs, docker exec и docker inspect использовались для исследования контейнера.
19. После stop контейнер отсутствовал в docker ps, но был виден в docker ps -a.
20. После rm контейнер исчез, а образ оставался в docker images.
21. Два экземпляра Nginx работали независимо; остановка одного не останавливала другой.
22. В индивидуальном задании контейнер student-web успешно создан, проверен и удалён.

14. Содержание отчёта

23. Титульные сведения: ФИО, группа, тема и цель.
24. Версии Docker Client/Server и подтверждение состояния службы.
25. Скриншот или текст успешного запуска hello-world.
26. Вывод docker ps -a и docker images с пояснением различий.
27. Скриншот страницы Nginx либо результат curl с кодом HTTP-ответа.
28. Фрагмент docker logs и сведения docker inspect (имя, статус, порты).
29. Результаты stop, start и rm с пояснением переходов состояния.
30. Подтверждение работы двух контейнеров с разными портами и диагностического эксперимента.
31. Результаты индивидуального задания и таблица не менее пяти команд Docker.
32. Ответы на контрольные вопросы и краткий вывод по работе.

15. Контрольные вопросы

33. Что такое Docker Engine и почему Docker Client не запускает контейнер напрямую?
34. Почему hello-world отсутствует в docker ps после успешного выполнения?
35. Чем образ отличается от контейнера, созданного из него?
36. Почему наличие nginx:stable в docker images не гарантирует работающий веб-сервер?
37. Что именно обозначают числа в -p 8080:80?
38. Почему localhost в Debian-ВМ и localhost на Windows указывают на разные системы?
39. Чем docker exec отличается от docker run?
40. Почему выход из оболочки, запущенной docker exec, не останавливает Nginx?
41. В чём различие docker stop, docker start и docker rm?
42. Почему удаление контейнера обычно не удаляет его исходный образ?
43. Почему два контейнера Nginx могут слушать порт 80 внутри себя одновременно?
44. Почему два контейнера не могут занять один и тот же опубликованный порт хоста?
45. Какие сведения docker logs и docker inspect помогают диагностировать неисправность?
46. Какие данные нужно сохранять при эксплуатации реальной базы данных, если контейнер удаляется?

16. Критерии оценивания

Оценивание лабораторной работы

Критерий	Максимум, баллов
Проверка Docker Engine и hello-world	10
Запуск Nginx и HTTP-проверка	20
Исследование логов, exec и inspect	15
Жизненный цикл контейнера	15
Два контейнера и диагностические эксперименты	15
Индивидуальное задание	15
Отчёт и контрольные вопросы	10
Итого	100

Рекомендуемая организация занятия: проверка среды — 15 минут; hello-world — 25 минут; Nginx и исследование — 45 минут; жизненный цикл и два контейнера — 40 минут; индивидуальное задание и диагностика — 35 минут; защита — 20 минут. Дополнительный опыт с конфликтом порта можно включать в блок самостоятельной диагностики.